GenAI Internals

Program 1

1. Explore pre-trained word vectors. Explore word relationships using vector arithmetic. Perform arithmetic operations and analyze results.

Program:

****

```
!pip install gensim numpy    #Installation

import gensim.downloader as api
import numpy as np

model = api.load("word2vec-google-news-300")

print("Vocabulary size:", len(model.key_to_index))

similar_words = model.most_similar("king", topn=5)

print("\nTop 5 words similar to 'king':")
for word, score in similar_words:
    print(f"{word} : {score}")

result = model.most_similar(positive=["king", "woman"],
                            negative=["man"],
                            topn=1)

print("\nResult of king - man + woman:")
print(result)
```

```
king_vec = model["king"]
man_vec = model["man"]
woman_vec = model["woman"]

new_vector = king_vec - man_vec + woman_vec

manual_result = model.similar_by_vector(new_vector, topn=5)

print("\nManual vector arithmetic result (Top 5):")
for word, score in manual_result:
    print(f"{word} : {score}")
```

****

Output:

```
Vocabulary size: 400000

Top 5 words similar to 'king':
prince : 0.7682328820228577
queen : 0.7507690787315369
son : 0.7020888328552246
brother : 0.6985775232315063
monarch : 0.6977890729904175

Result of king - man + woman:
[('queen', 0.7698540687561035)]

Manual vector arithmetic result (Top 5):
king : 0.8551837205886841
queen : 0.783441424369812
monarch : 0.6933802366256714
throne : 0.6833109855651855
daughter : 0.6809081435203552
```

Program 2

2. Use dimensionality reduction (e.g., PCA or t-SNE) to visualize word embeddings for Q 1. Select 10 words from a specific domain (e.g., sports, technology) and visualize their embeddings. Analyze clusters and relationships. Generate contextually rich outputs using embeddings. Write a program to generate 5 semantically similar words for a given input.

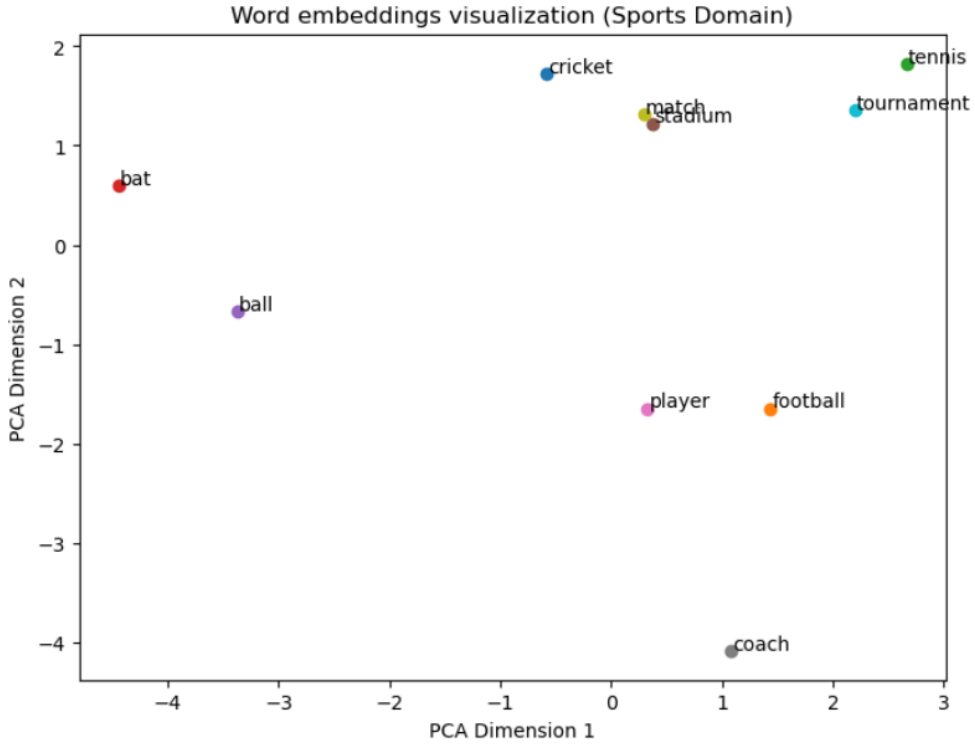
Program

****

```
!pip install gensim scikit-learn matplotlib    #installation

import gensim.downloader as api
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA

model = api.load("glove-wiki-gigaword-100")
print("model loaded successfully")

words = [
    "cricket", "football", "tennis", "bat",
    "ball", "stadium", "player",
    "coach", "match", "tournament"
]

word_vectors = []

for word in words:
```

```
    word_vectors.append(model[word])

word_vectors = np.array(word_vectors)

pca = PCA(n_components=2)
reduced_vectors = pca.fit_transform(word_vectors)

plt.figure(figsize=(8,6))

for i,word in enumerate(words):
    x=reduced_vectors[i,0]
    y=reduced_vectors[i,1]
    plt.scatter(x,y)
    plt.text(x+0.01,y+0.01,word)

plt.title("Word embeddings visualization (Sports Domain)")
plt.xlabel("PCA Dimension 1")
plt.ylabel("PCA Dimension 2")
plt.show()

word = input("Enter a word: ")
similar_words = model.most_similar(word,topn=5)

print("Top 5 similar words: ")

for w,score in similar_words:
    print(w,":",score)
**********************************************************************
```

Output -




```
Enter a word:  ball
Top 5 similar words:
kick : 0.7723649740219116
throw : 0.7400173544883728
balls : 0.7349199056625366
off : 0.7285565137863159
pitch : 0.7162730097770691
```

Program – 3

3. Train a custom Word2Vec model on a small dataset. Train embeddings on a domain-specific corpus (e.g., legal, medical) and analyze how embeddings capture domain-specific semantics.

Program

****

```
from gensim.models import Word2Vec
import pandas as pd
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

sentences = [
    ["doctor","treats","patient"],
    ["nurse","assists","doctor"],
    ["patient","takes","medicine"],
    ["surgeon","performs","surgery"],
    ["doctor","diagnoses","disease"],
    ["nurse","cares","patient"],
    ["hospital","treats","patients"],
    ["doctor","prescribes","medicine"],
    ["surgeon","works","hospital"],
    ["medicine","helps","patient"]
]

models = Word2Vec(sentences, vector_size = 100, window = 3, min_count = 1, workers = 4)
```

```
similar = models.wv.most_similar("patients",topn=5)
display(similar)

for word,score in similar:
    print(word,score)

word = input("Enter the word: ")

print(list(models.wv.index_to_key))

words = list(models.wv.index_to_key)
vectors = models.wv[words]
pca = PCA(n_components=2)
result = pca.fit_transform(vectors)
plt.figure(figsize=(8,6))
plt.scatter(result[:,0], result[:,1])

for i, word in enumerate(words):
    plt.annotate(word, xy=(result[i,0], result[i,1]))

plt.title("Word2Vec Word Embedding Visualization")
plt.show()
**********
```

Output – (next page)

[11]:
```
similar = models.wv.most_similar("patients",topn=5)
display(similar)
```

```
[('patient', 0.12300864607095718),
 ('assists', 0.08058694750070572),
 ('prescribes', 0.06548558920621872),
 ('doctor', 0.05433466657996178),
 ('treats', 0.016065239906311035)]
```

[10]:
```
for word,score in similar:
    print(word,score)
```

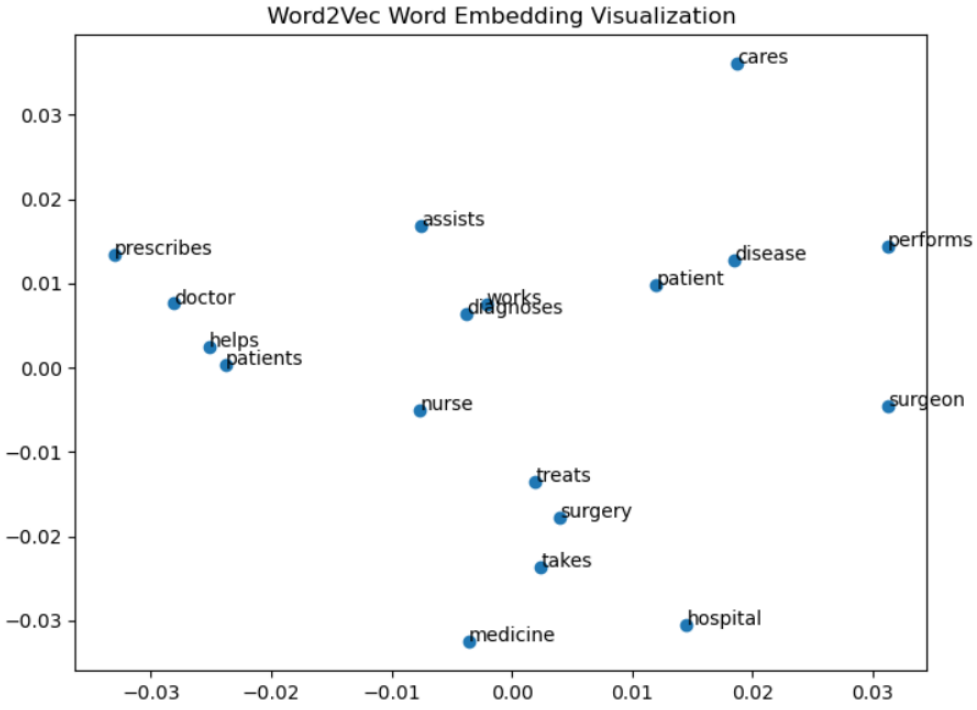
```
patient 0.12300864607095718
assists 0.08058694750070572
prescribes 0.06548558920621872
doctor 0.05433466657996178
treats 0.016065239906311035
```

[12]:
```
word = input("Enter the word: ")
```

```
Enter the word:  surgeon
```

[13]:
```
print(list(models.wv.index_to_key))
```

```
['doctor', 'patient', 'medicine', 'surgeon', 'treats', 'hospital', 'nurse', 'assists', 'takes', 'helps', 'works', 'surgery', 'diagnoses', 'disease', 'car
es', 'patients', 'prescribes', 'performs']
```



Program – 4

4. Use word embeddings to improve prompts for Generative AI model. Retrieve similar words using word embeddings. Use the similar words to enrich a GenAI prompt. Use the AI model to generate responses for the original and enriched prompts. Compare the outputs in terms of detail and relevance.

Program

****

```
!pip install torch transformers                #Installation

import gensim.downloader as api
from transformers import pipeline

model = api.load("glove-wiki-gigaword-100")

generator = pipeline("text-generation", model="gpt2")

original_prompt = "Explain cricket"

output1 = generator(original_prompt, max_length=50)

print("Original Prompt Output:\n")
print(output1[0]['generated_text'])

similar_words = model.most_similar("cricket", topn=5)

print("Similar words:\n")
for w, score in similar_words:
```

```
    print(w, score)

extra_words = [w for w, _ in similar_words]

enriched_prompt = original_prompt + " including " + ", ".join(extra_words)

print("\nEnriched Prompt:\n", enriched_prompt)

output2 = generator(enriched_prompt, max_length=80)

print("\nEnriched Prompt Output:\n")
print(output2[0]['generated_text'])
```

****

Output –

```
Original Prompt Output:

Explain cricket team's skillful cricketing, a team's knack for playing cricket and the potential of a strong spin-run. All the answers lie in the bats!

A player who's playing cricket at such a young age. Can
Similar words:

rugby 0.7827098369598389
twenty20 0.7362942099571228
england 0.7173773646354675
indies 0.715782105922699
cricketers 0.7132172584533691

Enriched Prompt:
 Explain cricket including rugby, twenty20, england, indies, cricketers

Enriched Prompt Output:

Explain cricket including rugby, twenty20, england, indies, cricketers and cricket in the English media. We are dedicated to making cricket an internatio
nal sport and offering an open competition for the youth interested in sport. In the spirit of the cricket community, this club's website has been in ope
ration for over 20 years and has offered both English and international cricket for over a 25 year run
```

Program – 5

5. Use word embeddings to create meaningful sentences for creative tasks. Retrieve similar words for a seed word. Create a sentence or story using these words as a starting point. Write a program that: Takes a seed word. Generates similar words. Constructs a short paragraph using these words.

Program

****

```
!pip install gensim transformers                    #Installation

import gensim.downloader as api
from transformers import pipeline

model=api.load("glove-wiki-gigaword-100")
print("Model loaded successfully")

generator= pipeline('text-generation', model="gpt2")

seed_word = input("Enter the seed word: ")

similar = model.most_similar(seed_word,topn=10)
for word in similar:
  print(word[0])

extra_words = [w for w,_ in similar]

sentence = " ".join(extra_words)
print(sentence)
```

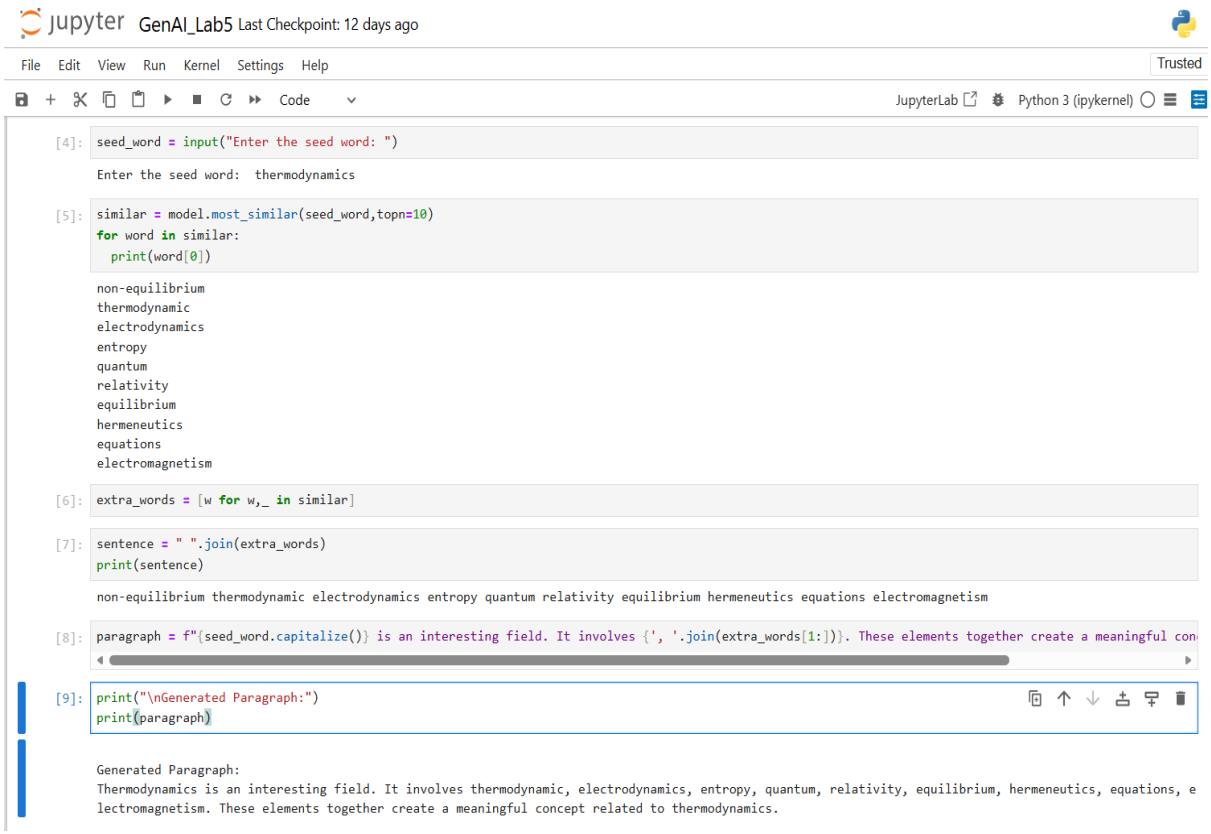
paragraph = f"{seed_word.capitalize()} is an interesting field. It involves {', '.join(extra_words[1:])}. These elements together create a meaningful concept related to {seed_word}."

print("\nGenerated Paragraph:")

print(paragraph)

****

Output –



Program – 6

6. Use a pre-trained Hugging Face model to analyze sentiment in text. Assume a real-world application, Load the sentiment analysis pipeline. Analyze the sentiment by giving sentences to input.

Program

****

```
!pip install transformers                    #Installation

from transformers import pipeline
sentiment = pipeline("sentiment-analysis")

Sentence = input("Enter the sentence: ")
result = sentiment(Sentence)
print(result)

sentence2 = input("Enter the sentence: ")
result2 = sentiment(sentence2)
print(result2)

sentence3 = input("Enter the sentence: ")
result3 = sentiment(sentence3)
print(result3)

positive_movie_review = input("Enter the review: ")
result4 = sentiment(positive_movie_review)
print(result4)
```

```
negative_movie_review = input("Enter a review: ")
result5 = sentiment(negative_movie_review)
print(result5)
```

****

Output –

```
[3]: Sentence = input("Enter the sentence: ")

Enter the sentence:  I am skeptical about this new movie about Winston Churchill

[6]: result = sentiment(Sentence)
     print(result)

[{'label': 'NEGATIVE', 'score': 0.9960130453109741}]

[7]: sentence2 = input("Enter the sentence: ")

Enter the sentence:  I don't like pineapple pizza, like fruit on pizza is simply infuriating. However, any other classic version is the best fast food o
f all time.

[8]: result2 = sentiment(sentence2)
     print(result2)

[{'label': 'POSITIVE', 'score': 0.9993008375167847}]

[9]: sentence3 = input("Enter the sentence: ")

Enter the sentence:  Memories of my childhood are vague and blurry and yet I remember the time I got lost in a fair for such a long time

[10]: result3 = sentiment(sentence3)
      print(result3)

[{'label': 'NEGATIVE', 'score': 0.9841581583023071}]

[12]: positive_movie_review = input("Enter the review: ")

Enter the review:  Darabont cut right to the heart of a flock of human birds trapped in a giant cage, under the watch of a corrupt warden and terrifying
ly sinister guard. It was serious, certain, and made you laugh. Freeman's legend has never shined brighter.

[13]: result4 = sentiment(positive_movie_review)
      print(result4)

[{'label': 'POSITIVE', 'score': 0.9974297881126404}]

[14]: negative_movie_review = input("Enter a review: ")

Enter a review:  Darabont, making his debut as a director, is swamped by ambition. There is just too much here -- too much plot, too many particulars, t
oo many prisoners vying for attention.

[15]: result5 = sentiment(negative_movie_review)
      print(result5)

[{'label': 'NEGATIVE', 'score': 0.9994131326675415}]
```

Program – 7

7. Summarize long texts using a pre-trained summarization model using Hugging face model. Load the summarization pipeline. Take a passage as input and obtain the summarized text.

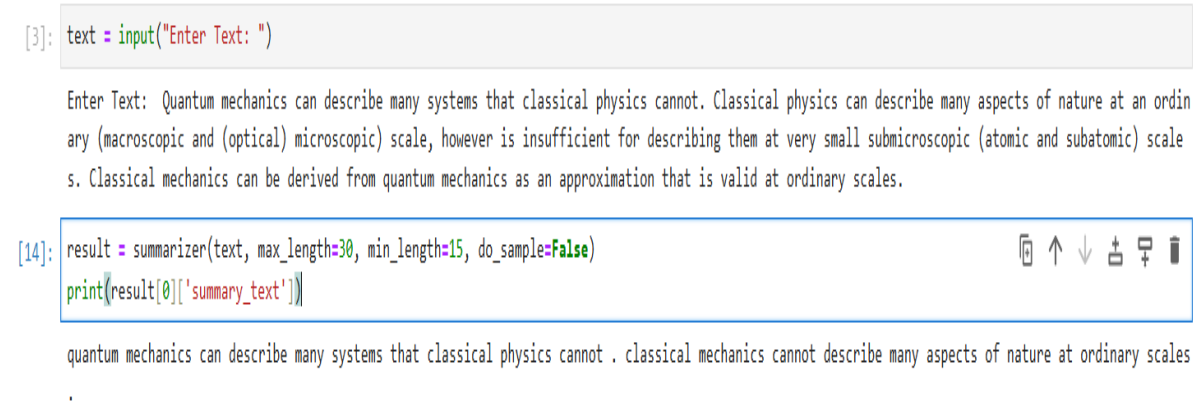
Program

****

```
!pip uninstall -y transformers                  #Uninstall
!pip install transformers==4.41.2               #Installation

from transformers import pipeline

summarizer = pipeline("summarization", model = "facebook/bart-large-cnn")
text = input("Enter Text: ")

result = summarizer(text, max_length=30, min_length=15, do_sample=False)
print(result[0]['summary_text'])
```

****

Output –



Program – 8

8. Install langchain, cohere (for key), langchain-community. Get the api key(By logging into Cohere and obtaining the cohere key). Load a text document from your google drive . Create a prompt template to display the output in a particular manner.

Program

****

```
!pip install langchain cohere langchain-community langchain-cohere -q
#Installation

from google.colab import drive
drive.mount('/content/drive')

file_path = "/content/drive/MyDrive/GenAI.txt"

with open(file_path, "r") as f:
    document_content = f.read()

print("Document loaded successfully!")
print(f"\nPreview (first 300 chars):\n{document_content[:300]}")

import os

COHERE_API_KEY= " "                    #paste the cohere API Key
os.environ["COHERE_API_KEY"] = COHERE_API_KEY
```

```
print("Cohere API Key set successfully!")
from langchain_cohere import ChatCohere
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser

llm = ChatCohere(
    cohere_api_key=COHERE_API_KEY,
   model="command-r-plus-08-2024",
    temperature=0.3,
)

template = """
You are a helpful assistant. Below is a document provided by the user.

Document Content:
{document_content}

Based on the document above, answer the following question clearly and concisely:
{user_question}

Format your response as:
----------------------------
 Question  : {user_question}
 Answer    : <your answer here>
 Source    : Based on the provided document
----------------------------
"""
```

```
prompt = PromptTemplate(
    input_variables=["document_content", "user_question"],
    template=template,
)

chain = prompt | llm | StrOutputParser()

user_question = input(" Enter your question about the document: ").strip()

print("\n Fetching answer...\n")
response = chain.invoke({
    "document_content": document_content,
    "user_question": user_question,
})

print(response)
```

****

Output –

```
)

chain = prompt | llm | StrOutputParser()

user_question = input(" Enter your question about the document: ").strip()

print("\n Fetching answer...\n")
response = chain.invoke({
    "document_content": document_content,
    "user_question": user_question,
})

print(response)
```

```
 Enter your question about the document: when was quantum thermodynamics developed and what are the key components of it

 Fetching answer...

----------------------------
 Question  : when was quantum thermodynamics developed and what are the key components of it
 Answer    : Quantum thermodynamics was developed in the early 20th century, with Albert Einstein's 1905 paper on the quantization of light being a significant milestone. Th

Key components of quantum thermodynamics include:
- The relationship between thermodynamics and quantum mechanics, particularly the quantization of light and matter.
- The emergence of thermodynamic laws from quantum mechanics, focusing on dynamical processes out of equilibrium.
- The relevance to individual quantum systems, as opposed to statistical ensembles.
- The connection with open quantum systems, where the system and its environment (bath) are considered together.
- The use of reduced dynamics and the Lindblad equation to describe the evolution of open quantum systems.
- The application of the Heisenberg picture to link quantum thermodynamic observables.
 Source    : Based on the provided document
----------------------------
```

Program – 9

9. Take the Institution name as input. Use Pydantic to define the schema for the desired output and create a custom output parser. Invoke the Chain and Fetch Results. Extract the below Institution related details from Wikipedia: The founder of the Institution. When it was founded. The current branches in the institution . How many employees are working in it. A brief 4-line summary of the institution.

Program

****

```
!pip install langchain cohere langchain-community langchain-cohere wikipedia pydantic -q                    #Installation

import os

COHERE_API_KEY = "uxNxTg23XraCNLMvKhkZ41yQ8SY"
os.environ["COHERE_API_KEY"] = COHERE_API_KEY

print(" Cohere API Key set successfully!")

from pydantic import BaseModel, Field
from typing import List

class InstitutionInfo(BaseModel):
   institution_name : str       = Field(description="Full official name of the institution")
   founder          : str       = Field(description="Founder or founders of the institution")
```

```
    founded_year     : str       = Field(description="Year or date the institution was
founded")
    branches         : List[str] = Field(description="List of current branches of the
institution")
    employees        : str       = Field(description="Approximate number of
employees")
    summary          : str       = Field(description="A brief 4-line summary of the
institution")

print(" Pydantic Schema defined!")
print("\n Fields:")
for field, info in InstitutionInfo.model_fields.items():
    print(f"   • {field:20s} → {info.description}")

from langchain_core.output_parsers import BaseOutputParser
import json, re

class InstitutionOutputParser(BaseOutputParser[InstitutionInfo]):

    def parse(self, text: str) -> InstitutionInfo:

        text = re.sub(r"```(?:json)?", "", text).strip()

        json_match = re.search(r'\{.*\}', text, re.DOTALL)
        if not json_match:
            raise ValueError(f"No valid JSON found in LLM response:\n{text}")

        raw = json.loads(json_match.group())
```

```
        branches = raw.get("branches", [])
        if isinstance(branches, str):
            branches = [b.strip() for b in re.split(r",|;|\n", branches) if b.strip()]

        return InstitutionInfo(
            institution_name = raw.get("institution_name", "N/A"),
            founder          = raw.get("founder",          "N/A"),
            founded_year     = raw.get("founded_year",     "N/A"),
            branches         = branches,
            employees        = raw.get("employees",        "N/A"),
            summary          = raw.get("summary",          "N/A"),
        )

    @property
    def _type(self) -> str:
        return "institution_output_parser"

print(" Custom Output Parser ready!")

from langchain_cohere import ChatCohere
from langchain_core.prompts import PromptTemplate

llm = ChatCohere(
    cohere_api_key=COHERE_API_KEY,

    model="command-r-08-2024",
    temperature=0.1,
    max_tokens=800,
)
```

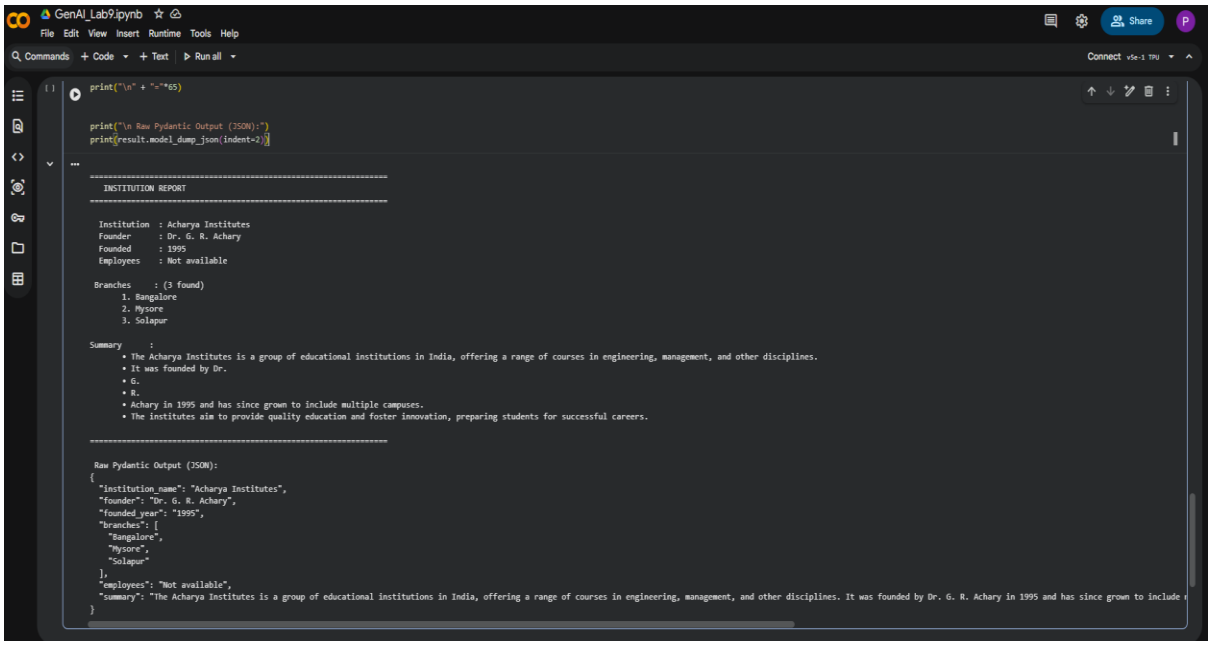
```
template = """
You are a research assistant. Extract details about this institution from your knowledge.

Institution: {institution_name}

Return ONLY a raw JSON object with NO markdown, NO code fences, NO extra text.

{{
  "institution_name" : "<full official name>",
  "founder"          : "<name(s) of founder(s)>",
  "founded_year"     : "<year founded>",
  "branches"         : ["<branch 1>", "<branch 2>", "<branch 3>"],
  "employees"        : "<number of employees>",
  "summary"          : "<Line 1. Line 2. Line 3. Line 4.>"
}}

If any value is unknown, write "Not available".
"""

prompt = PromptTemplate(
    input_variables=["institution_name"],
    template=template,
)

parser = InstitutionOutputParser()
chain  = prompt | llm | parser
```

```
print(" Chain ready!")

institution_name = input("Enter Institution Name: ").strip()

print(f"\nSearching Wikipedia knowledge for: '{institution_name}' ...")
print("Please wait...\n")

result: InstitutionInfo = chain.invoke({
    "institution_name": institution_name
})

print("Results fetched successfully!")

print("\n" + "="*65)
print(f"   INSTITUTION REPORT")
print("="*65)
print(f"\n  Institution  : {result.institution_name}")
print(f"  Founder      : {result.founder}")
print(f"  Founded      : {result.founded_year}")
print(f"  Employees    : {result.employees}")

print(f"\n Branches     : ({len(result.branches)} found)")
for i, branch in enumerate(result.branches, 1):
    print(f"       {i}. {branch}")

print(f"\nSummary      :")
sentences = [s.strip() for s in result.summary.split(".") if s.strip()]
for sentence in sentences:
```

```
    print(f"      • {sentence}.")

print("\n" + "="*65)

print("\n Raw Pydantic Output (JSON):")
print(result.model_dump_json(indent=2))
```

****

Output:



Program – 10

10. Build a chatbot for the Indian Penal Code. We'll start by downloading the official Indian Penal Code document, and then we'll create a chatbot that can interact with it. Users will be able to ask questions about the Indian Penal Code and have a conversation with it.

Program

****

```
!pip install pypdf                          #Installation

from google.colab import files
print("Upload your ipc.pdf")
uploaded = files.upload()

from pypdf import PdfReader
reader = PdfReader("ipc.pdf")

text = ""
for page in reader.pages:
    content = page.extract_text()
    if content:
        text += content

text = text.replace("\n", " ")
print("Text length:", len(text))
print(text[:500])

docs = text.split("Section")
```

```
docs = ["Section " + d.strip() for d in docs if len(d.strip()) > 30]
print("Total sections:", len(docs))

from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity

model = SentenceTransformer('all-MiniLM-L6-v2')
doc_embeddings = model.encode(docs, show_progress_bar=True)

import re

def chatbot(query):
    query_lower = query.lower()
    match = re.search(r'section\s*(\d+)', query_lower)

    if match:
        section_num = match.group(1)
        for doc in docs:
            if f"Section {section_num}" in doc:
                return doc
        return f"Section {section_num} not found."

    query_embedding = model.encode([query])
    similarity = cosine_similarity(query_embedding, doc_embeddings)[0]
    top_index = similarity.argmax()
    return docs[top_index]

print("\nIPC Chatbot Ready! Type 'exit' to stop\n")
```

```
while True:
    q = input("You: ")
    if q.lower() == "exit":
        break
    print("Bot:", chatbot(q), "\n")
```

****

Output: